

GDD — Sistema de Habilidades Elementais

Projeto multiplayer, host = peer 1, dev solo, horizonte de ~1 ano

0. Filosofia de design (leia isso antes do resto)

O maior risco desse projeto não é "vai ficar bonito", é **explosão combinatória**. Com 5 elementos x 7 slots x interações contextuais (projétil-projétil, projétil-decal, decal-decal, carregado-vs-solto), se você programar cada combinação na mão dentro do script de cada habilidade, em 3 meses você vai ter 40 arquivos com `(if element == FIRE and other == EARTH and context == DECAL: ...)` espalhados e ninguém — nem você — vai conseguir dar manutenção nisso.

Por isso a regra de ouro deste GDD é: **nenhuma habilidade sabe com o que ela reage**. Toda habilidade só sabe "eu sou fogo, sou um projétil, tenho tanto de dano". Quem decide o que acontece quando fogo toca terra é uma **tabela de reações centralizada** (a "Tabela Alquímica"), parecida com uma tabela de dados, não código espalhado. Isso é o que vai permitir você adicionar o elemento 6 no mês 8 sem reescrever nada.

Isso também é o que separa este documento de "gambiarra de game jam": tudo aqui é dado (`Resource` do Godot), não lógica hardcoded. Você vai editar `.tres` no Inspector, não vasculhar scripts.

1. Arquitetura de rede (recap + ajuste necessário)

Você já tem o padrão certo com a `ball.gd`: cliente pede via RPC pro servidor (`id 1`), servidor instancia no `spawn_container` (replicado via `MultiplayerSpawner`), e só o servidor resolve colisão (`is_multiplayer_authority()` / `multiplayer.is_server()`).

Todo o sistema de habilidades vai seguir exatamente esse fluxo, sem exceção:

```
Cliente aperta Q
-> valida cooldown localmente (só pra UI/feedback, não é fonte da verdade)
-> RPC "any_peer" pro servidor: cast_ability(slot, pos, dir)

Servidor
-> valida cooldown de verdade (nunca confie no cliente)
-> instancia o efeito no spawn_container (replica sozinho)
-> toda a lógica de colisão/reação roda só no servidor
-> quando algo "visualmente" precisa acontecer pra todo mundo
    (explosão, mudança de shader, som), dispara RPC "call_local"
```

Um erro comum de iniciante é achar que "reação visual" precisa de sincronização constante. Não precisa. Trocar o shader de uma boulder pra "lava" é um **evento único** (aconteceu ou não aconteceu), então um RPC disparado uma vez e replicado (`call_local`) resolve — você não precisa de um `MultiplayerSynchronizer` rodando toda hora pra isso.

2. Os 5 Elementos e os 7 Slots

2.1 Enum central

gdscript

```
# elements.gd (autoload)
extends Node

enum Element { FIRE, WATER, EARTH, AIR, ELECTRIC }

enum Slot {
    AUTO_ATTACK,      # LMB
    SPECIAL_ATTACK,    # RMB
    PROJECTILE,        # Q
    SPECIAL_SKILL,     # E
    DASH,              # SHIFT
    JUMP,              # SPACE
    ULTIMATE,          # R
}
```

2.2 Ability como Resource (dado, não código)

Cada uma das 7 habilidades é um `.tres`, não uma classe nova. A diferença de comportamento vem de **qual cena de efeito** ela instancia e **quais tags** ela carrega, não de scripts diferentes por habilidade.

gdscript

```
# ability.gd
class_name Ability
extends Resource

@export var ability_name: String
@export var slot: Elements.Slot
@export var element: Elements.Element
@export var cooldown: float = 1.0
@export var resource_cost: float = 0.0 # mana/stamina, se um dia existir
@export var effect_scene: PackedScene # o que essa habilidade instancia
@export var icon: Texture2D
@export var tags: Array[String] = [] # ex: ["projectile", "aoe", "charge"]
```

2.3 ElementKit — o "personagem" é o elemento

Como você quer que jogadores possam ter o **mesmo elemento**, o kit de 7 habilidades não pertence ao jogador, pertence ao elemento. Isso significa: escolher elemento = já vem com as 7 habilidades prontas. Customização de personagem (skin, cor de partícula, nome) fica separada disso.

gdscript

```
# element_kit.gd
class_name ElementKit
extends Resource

@export var element: Elements.Element
@export var display_name: String
@export var abilities: Array[Ability] = [] # sempre 7, uma por Slot
```

No Player, isso fica assim:

gdscript

```
# Player.gd
@export var element_kit: ElementKit
var cooldown_until := {}

const SLOT_INPUT = {
    Elements.Slot.AUTO_ATTACK: "attack_lmb",
    Elements.Slot.SPECIAL_ATTACK: "attack_rmb",
    Elements.Slot.PROJECTILE: "ability_q",
    Elements.Slot.SPECIAL_SKILL: "ability_e",
    Elements.Slot.DASH: "ability_shift",
    Elements.Slot.JUMP: "ability_space",
    Elements.Slot.ULTIMATE: "ability_r",
}

func _process(_delta):
    if not is_multiplayer_authority(): return
    for slot in SLOT_INPUT:
        if Input.is_action_just_pressed(SLOT_INPUT[slot]):
            try_cast(slot)

func try_cast(slot: Elements.Slot):
    var ability = _get_ability(slot)
    if not ability: return
    var now = Time.get_ticks_msec() / 1000.0
    if cooldown_until.get(slot, 0.0) > now:
        return
    cooldown_until[slot] = now + ability.cooldown
    Global.cast_ability.rpc_id(1, element_kit.element, slot, global_position, g

func _get_ability(slot: Elements.Slot) -> Ability:
    for a in element_kit.abilities:
        if a.slot == slot:
            return a
    return null
```

Por que isso é sustentável: pra criar o elemento Gelo daqui a 6 meses, você não escreve

nenhum script novo de player — você cria 7 `Ability.tres`, aponta pra 7 cenas de efeito, empacota num `ElementKit.tres` novo. Zero código.

3. A "Tabela Alquímica" (Reaction Database)

Esse é o coração do sistema e o que resolve seus dois exemplos (Boulder+Fire e Earth Jump+Fire).

3.1 Por que não pode ser "if/else" nos scripts dos efeitos

Seus exemplos mostram que a reação **não é sempre "destrua os dois e exploda"**. Às vezes é:

- **Infusão**: a boulder não morre, ela *muda* (ganha shader de lava, passa a emitir partículas de fogo, e ao cair gera explosão B em vez de explosão A).
- **Interrupção + atraso**: a cratera de terremoto não reage na hora — ela para o dano periódico, muda de estado, e só depois de 2s explode.

Se isso estiver no código de cada cena, você não consegue reaproveitar a lógica. Por isso a reação é **dado configurável com um "tipo de resultado"**.

3.2 `ReactionRule`

gdscript

```
# reaction_rule.gd
class_name ReactionRule
extends Resource

enum Outcome {
    DESTROY_AND_SPAWN, # os dois somem, nasce um efeito novo (explosão simples)
    INFUSE,             # A absorve o elemento de B, muda visual/comportamento,
    DELAYED_TRIGGER,    # nada acontece na hora, um timer começa, resultado só d
    MUTATE_DECAL,       # um decal no chão muda de estado (ex: cratera -> cratera

}

@export var element_a: Elements.Element
@export var element_b: Elements.Element
@export var tag_a: String = "" # "" = qualquer contexto, ou "projectile"/"decal"
@export var tag_b: String = ""

@export var outcome: Outcome
@export var result_scene: PackedScene # explosão, ou o "estado infundido" da
@export var delay: float = 0.0 # usado em DELAYED_TRIGGER
```

3.3 `ReactionDatabase` (autoload)

gdscript

```
# reaction_database.gd (autoload)
extends Node

@export var rules: Array[ReactionRule] = [] # populado no Inspector com vários

func find_rule(elem_a, tag_a, elem_b, tag_b) -> ReactionRule:
    for rule in rules:
        var match_direct = (rule.element_a == elem_a and rule.element_b == elem_b
            and (rule.tag_a == "" or rule.tag_a == tag_a)
            and (rule.tag_b == "" or rule.tag_b == tag_b))
        var match_swapped = (rule.element_a == elem_b and rule.element_b == elem_a
            and (rule.tag_a == "" or rule.tag_a == tag_b)
            and (rule.tag_b == "" or rule.tag_b == tag_a))
        if match_direct or match_swapped:
            return rule
    return null
```

Com isso, adicionar "Água apaga Fogo" ou "Elétrico + Água = choque em área" vira **criar um .tres** novo, não escrever GDScript.

3.4 Interface comum: **ElementalCarrier**

Toda coisa que carrega um elemento e pode reagir (projétil, decal, área carregada) estende ou implementa a mesma interface mínima:

```
gdscript

# elemental_carrier.gd
class_name ElementalCarrier
extends Node

var element: Elements.Element
var tag: String = "projectile" # "projectile", "decal", "charge"
var owner_peer_id: int
var infused_with: Elements.Element = -1 # -1 = não infundido
```

Efeitos concretos (boulder, fireball, cratera) **estendem** essa classe (ou compõem ela como um componente) além da física deles (**RigidBody3D**, **Area3D**, etc). O importante é que a resolução de colisão sempre pergunta pro **ReactionDatabase**, nunca decide sozinha.

4. Andando pelos seus dois exemplos com o framework

4.1 Boulder + Fireball (infusão em pleno ar)

1. Jogador 1 usa Q (terra) → servidor instancia **boulder.tscn**. Ela estende **ElementalCarrier** com **element = EARTH**, **tag = "projectile"**.
2. Jogador 2 usa Q (fogo) → servidor instancia **fireball.tscn**, **element = FIRE**, **tag = "projectile"**.
3. As duas têm uma **Area3D** cujo **area_entered** chama uma função **genérica** (não

específica de fogo ou terra):

gdscript

```
func _on_area_entered(other: Area3D):
    if not multiplayer.is_server(): return
    if not (other is ElementalCarrier): return
    if get_instance_id() < other.get_instance_id(): return # evita disparo dupl

    var rule = ReactionDatabase.find_rule(element, tag, other.element, other.tag)
    if not rule:
        return
    ReactionResolver.resolve(rule, self, other)
```

4. A regra `EARTH(projectile) + FIRE(projectile)` está cadastrada como `Outcome.INFUSE`. O `ReactionResolver` então:
 - não dá `queue_free()` na boulder.
 - seta `boulder.infused_with = FIRE`.
 - troca o `shader_material` dela pra variante lava (você expõe um `ShaderMaterial` com parâmetro `infusion_progress` e só anima ele).
 - liga um `GPUParticles3D` de fogo que já existe desligado na cena da boulder (deixe pré-instanciado e oculto — é mais barato que instanciar em runtime).
 - dá `queue_free()` só na fireball.
5. Quando a boulder finalmente colide com o chão, o script dela pergunta pro próprio estado: `if infused_with == FIRE: spawn(explosion_B) else: spawn(explosion_A)`.

Ponto chave de sustentabilidade: a boulder nunca precisou saber que fogo existe. Ela só expõe `infused_with` e reage a esse campo genérico no impacto. Isso funcina igual se amanhã você criar "Boulder + Elétrico" (ex: explosão C) — é só adicionar a regra e o `if`/`match` no impacto.

4.2 Earth Jump (carregado) + Fireball no decal (interrupção com atraso)

1. Jogador segura Space → carrega, solta → servidor instancia área de impacto: dano periódico (`Timer` tickando `take_damage` em quem estiver dentro) + spawna um decal `crater_decal.tscn` com `element = EARTH`, `tag = "decal"`.
2. O decal é ele mesmo um `ElementalCarrier` (`tag = "decal"`) com uma `Area3D` cobrindo a área da cratera.
3. Outro jogador (ou o mesmo) joga uma fireball em cima. A fireball colide com a `Area3D` do decal → mesmo fluxo genérico do item 4.1 → `ReactionDatabase.find_rule(EARTH, "decal", FIRE, "projectile")`.
4. Essa regra é `Outcome.DELAYED_TRIGGER`, com `delay = 2.0` e `result_scene = mega_explosion.tscn`.
5. O `ReactionResolver`:
 - imediatamente para o `Timer` de dano periódico do decal (`crater.stop_periodic_damage()`) — um método público simples que qualquer decal expõe).

- troca o shader do decal pra "prestes a explodir" (ex: brilho pulsante) via RPC `call_local` de uma vez só.
- cria um `Timer` de 2s; ao estourar, spawna `mega_explosion.tscn` no servidor e faz `queue_free()` no decal.

Ponto chave: o decal só precisa expor 2 ganchos genéricos — `stop_periodic_damage()` e um sinal/estado de "infundido" — e o resolver decide o resto via dado. Nenhuma lógica de fogo dentro do script do decal de terra.

5. Decals no chão (efeitos persistentes de terreno)

Todo decal (fogo no chão, cratera, poça d'água) é uma cena com essa forma:

```
gdscript

# ground_decal.gd
class_name GroundDecal
extends Area3D # (também é um ElementalCarrier, via composição ou herança)

@export var element: Elements.Element
var tag := "decal"
var owner_peer_id: int
var infused_with: Elements.Element = -1

@onready var tick_timer: Timer = $TickTimer # dano/slow periódico, se aplicável
@onready var shape: CollisionShape3D = $CollisionShape3D
@onready var particles: GPUParticles3D = $Particles
@onready var mesh: MeshInstance3D = $Mesh # com ShaderMaterial único

func _ready():
    add_to_group("ElementalCarriers")
    area_entered.connect(_on_area_entered)
    body_entered.connect(_on_body_entered) # pra dano em CharacterBody3D

func stop_periodic_damage():
    tick_timer.stop()

func _on_body_entered(body):
    if not multiplayer.is_server(): return
    if body.has_method("take_damage"):
        # aplica efeito de status conforme o elemento (queimadura, lentidão, mo
    pass
```

Isso te dá um padrão **único** pra "fogo no chão" (deixado por um dash de fogo), "cratera" (deixada por earth jump) e "poça d'água" — todos a mesma classe base, diferenciados só por `element` e pelos parâmetros de shader/timer no Inspector.

6. Dano, knockback e status: padronizando pros inimigos

Hoje seu `target.gd` tem `take_damage(damage, source)`. Isso não escala pra knockback, status de elemento (queimadura, lentidão, choque) nem pra "quem" está aplicando o efeito (jogador vs decal vs explosão). Padronize a assinatura logo, antes de ter 10 lugares chamando `take_damage` de formas diferentes:

gdscript

```
# damageable.gd — não precisa ser Resource, é só um contrato de método.  
# Qualquer StaticBody3D/CharacterBody3D/enemy implementa isso:
```

```
func take_damage(info: DamageInfo) -> void:  
    pass
```

gdscript

```
# damage_info.gd  
class_name DamageInfo  
extends RefCounted  
  
var amount: float  
var source_peer_id: int  
var element: Elements.Element  
var knockback_dir: Vector3 = Vector3.ZERO  
var knockback_force: float = 0.0  
var status_effect: String = "" # "burn", "slow", "shock", "" = nenhum  
var status_duration: float = 0.0
```

E um componente reaproveitável pra HP, pra não reescrever a mesma lógica em cada inimigo/target:

gdscript

```
# health_component.gd
class_name HealthComponent
extends Node

signal died(source_peer_id)
signal hit(info: DamageInfo)

@export var max_health := 100.0
var health := max_health

func apply(info: DamageInfo):
    if not multiplayer.is_server(): return
    health -= info.amount
    hit.emit(info)
    if health <= 0:
        died.emit(info.source_peer_id)
```

Toda habilidade, decal ou explosão que causa dano monta um `DamageInfo` e chama `body.take_damage(info)`. Knockback é aplicado de forma padronizada no corpo `(CharacterBody3D.velocity += info.knockback_dir * info.knockback_force)`, ou `(RigidBody3D.apply_central_impulse(...))`, então você escreve essa física **uma vez**, num método utilitário, não em cada habilidade.

7. Estrutura de pastas sugerida

```

res://
  autoloads/
    global.gd
    network.gd
    elements.gd
    reaction_database.gd
    reaction_resolver.gd
  data/
    element_kits/          # ElementKit .tres (fire_kit.tres,
earth_kit.tres...)
    abilities/             # Ability .tres (fire_q.tres, earth_jump.tres...)
    reaction_rules/        # ReactionRule .tres
(earth_fire_projectile.tres...)
    entities/
      player/
      enemies/
    effects/
      projectiles/         # fireball.tscn, boulder.tscn...
      decals/              # crater_decal.tscn, fire_ground.tscn,
wet_puddle.tscn
      explosions/          # explosion_a.tscn, explosion_b.tscn,
mega_explosion.tscn
    world/
      open_world.tscn
    lobby/
      lobby.tscn
    ui/

```

Separar `data/` de `effects/` (cenas) é o que permite você balancear cooldown/dano só editando `.tres` sem tocar em código nem reexportar lógica.

8. Lobby físico (fase futura, já pensando na integração)

Quando chegar a hora, o Lobby é só **outra cena de mundo**, com o mesmo `spawn_container` + `MultiplayerSpawner`, só que:

- Tem "pads" de seleção de elemento: `Area3D` que, ao jogador entrar, troca o `element_kit` dele via RPC validada no servidor (nunca deixe o cliente setar o próprio elemento sem validação).
- Tem um dummy (igual seu `target.gd`), só que com HP infinito/regenerando, pra testar dano sem "matar" o alvo de teste).
- Seleção de mapa/dificuldade é uma UI que, ao confirmar, faz o host chamar algo como `Global.load_world.rpc(map_id, difficulty)` — troca de cena sincronizada pra todos os peers.
- Um `GameState` autoload simples guarda um enum `{ LOBBY, WORLD }` pra você saber onde cada player está e não deixar `spawn_container` do lobby e do mundo aberto se misturarem (**use containers diferentes ou raiz de cena diferente pra cada fase**, senão o `MultiplayerSpawner` de um vaza pro outro).

Isso é 100% compatível com a arquitetura de habilidades acima — o dummy do lobby só precisa ter um `HealthComponent` também, então testar uma habilidade lá já usa o mesmo pipeline de dano do mundo aberto.

9. Roadmap sugerido (1 ano, dev solo)

Fase	Duração aprox.	Entregável
M0 — Fundação de rede	2-3 sem	Corrigir bug de IP fixo, feedback de conexão, <code>MultiplayerSpawner</code> revisado
M1 — Pipeline de 1 elemento completo	4-6 sem	<code>Ability</code> , <code>ElementKit</code> , os 7 slots funcionando fim-a-fim com 1 elemento só (ex: Fogo) — antes de escalar, prove que o pipeline funciona
M2 — Framework de reação + decal	4-6 sem	<code>ReactionRule</code> , <code>ReactionDatabase</code> , <code>GroundDecal</code> genérico, provar os 2 casos de exemplo (Boulder+Fire, Earth Jump+Fire) com 2 elementos
M3 — Elementos restantes	6-8 sem	Terra, Água, Ar, Elétrico com os 7 slots cada + matriz de reações completa (ou subconjunto relevante)
M4 — Dano/knockback padronizado + inimigos básicos	4-6 sem	<code>DamageInfo</code> , <code>HealthComponent</code> , 1-2 tipos de inimigo usando o mesmo contrato
M5 — Lobby	4 sem	Seleção de elemento, dummy, seleção de mapa/dificuldade
M6 — Mundo aberto + progressão	8-10 sem	Spawns de inimigos, desbloqueáveis, customização visual
M7 — Polimento e balanceamento	resto do ano	Netcode stress test, ajuste de dano/cooldown, bugs

O ponto crítico do roadmap pra um dev solo: **não tente fazer 5 elementos ao mesmo tempo em M1**. Prove o pipeline com 1 elemento e o framework de reação com 2, porque é ali que moram os bugs de arquitetura. Depois de M2 validado, adicionar elemento 3, 4 e 5 deve ser **só preencher dados**, não escrever sistema novo — se isso não for verdade quando você chegar em M3, é sinal de que algo em M1/M2 ainda está hardcoded demais e vale parar pra refatorar antes de continuar.

10. Checklist de "isso é dado, não código" (revisão rápida antes de cada nova feature)

Antes de escrever uma nova habilidade ou elemento, pergunte:

☐ Dá pra criar isso só com um `.tres` novo, sem tocar em `.gd`?

- ☐ O efeito sabe reagir sozinho, ou ele pergunta pro `ReactionDatabase`?
- ☐ O dano/knockback passa por `DamageInfo`, ou tem um `take_damage(int, int)` solto em algum lugar?
- ☐ Se eu remover esse elemento amanhã, quantos arquivos eu preciso editar? (resposta ideal: 0 scripts, só remover os `.tres`)

Se a resposta a qualquer uma dessas for "não", é sinal de acoplamento que vai custar caro lá pelo mês 8.